AI-Powered Development

From LLMs to Agents — How Everything Connects

AI LLM API Providers Agents MCP

OVERVIEW

The Big Picture

These concepts form a stack. Each layer builds on the one below it.

🧠 **AI (Artificial Intelligence)**
The broad field — machines that simulate intelligence

▼

📖 **LLM (Large Language Model)**
A type of AI — text prediction engines trained on massive data

▼

🔌 **API (Application Programming Interface)**
How you talk to an LLM programmatically

▼

🏢 **API Providers**
OpenRouter, Kilo Code, OpenAI

▼

🤖 **AI Agents**
Roo Code, Cline, Claude Code

▼

🔗 **MCP (Model Context Protocol)**
Standard way for agents to connect to tools & data

LAYER 1

🧠 AI — Artificial Intelligence

AI is the **broadest term** here. It means any system that mimics or simulates human intelligence.

- Recognizing images (computer vision)
- Playing chess or Go (game AI)
- Recommending videos (recommendation engines)
- Understanding and generating language (LLMs)

> **ANALOGY**
> *Think of AI like* ***"Vehicles."*** *It's the category. Cars, trucks, motorcycles — they're all vehicles, but they work differently. LLMs are one type of AI vehicle.*

Not all AI is LLMs. Spam filters and self-driving cars are AI too.

LAYER 2

📖 LLM — Large Language Model

An LLM is a specific type of AI — one trained to **predict the next word** in a sequence. Train it on enough data, and it starts to *seem* like it understands.

Tokens

The smallest unit of text an LLM processes. Not exactly words — can be a word, part of a word, or punctuation.

1000 tokens ≈ 750 words

Context Window

The maximum text the LLM can "hold in memory" at once — your prompt + its response combined.

Like working memory

Temperature

Controls randomness. Low (0.1) = predictable. High (0.9) = creative.

Textbook vs. Creative dial

LAYER 2 — DETAILS

📖 Common LLMs

MODEL	MADE BY	STRENGTHS	CONTEXT WINDOW
GPT-4 / GPT-5	OpenAI	General purpose, strong reasoning	8K – 128K
Claude 4 / Sonnet	Anthropic	Long context, code, safety	~200K
Gemini 2.5	Google	Multimodal, generous free tier	~1M
Llama 3 / 4	Meta	Open source, self-hostable	8K – 128K

KEY POINT

*You don't need to run these models yourself. **API Providers** host them on their servers. You just send requests.*

LAYER 3

🔌 API — Application Programming Interface

An API is how **programs talk to each other**. Instead of you typing into a chat window, your *code* sends a message to the LLM and gets a response back.

```
// Your code sends a request to the LLM POST https://api.openai.com/v1/chat/completions Body: { "model": "gpt-4", "messages": [ { "role": "user", "content": "Hello!" } ] } // Response comes back { "choices": [{ "message": { "content": "Hi there!" } }] }
```

> **ANALOGY**
>
> *An API is like a **phone call**. You dial a number (the endpoint), say something (the request), and get a reply (the response). The LLM is on the other end of the line.*

LAYER 4A

🏢 API Providers

These are the companies that **host the LLMs** and let you access them via API. You don't run the model — they do.

PROVIDER	WHAT THEY OFFER	FREE TIER?
OpenRouter	One API, many models (GPT, Claude, open-source)	Yes — several free models
Kilo Code	Developer-focused AI API	Free tier available
OpenAI	GPT models	Limited credits for new accounts
Google	Gemini models	Generous free tier

Recommendation: Start with OpenRouter. One key, many models, free options available.

LAYER 4B

🤖 AI Agents

An agent is AI that doesn't just answer questions — it does things. It can plan, use tools, and iterate until the job is done.

> **ANALOGY**
>
> ***Chatbot*** *= asking someone for directions.*
> ***Agent*** *= asking someone to go deliver a package.*

You give a task → Agent plans steps → Uses tools → Checks work → Delivers result

LAYER 4B — SKILLS

🎓 Agent Skills

Skills are **pre-built instruction sets** that teach an agent how to behave in specific scenarios.

> **ANALOGY**
>
> *If **tools** are what an agent can touch, skills are what an agent knows. Like the difference between having a wrench (tool) and knowing plumbing (skill).*

📋 Custom Instructions

Define how the agent should respond — tone, format, level of detail, coding style preferences.

🎯 Task-Specific Skills

"Always write tests first", "Explain before coding", "Ask before deleting files" — behavioral rules.

📁 .clinerules / AGENTS.md

Project-specific skill files that live in your repo. The agent reads them automatically on every session.

🔄 Skill Modes

Switch between "architect mode" (plan first) and "builder mode" (code immediately) depending on the task.

```
# Example: .clinerules file Always explain your reasoning before writing
code. Write tests before implementation (TDD). Never remove existing
functionality without asking. Use TypeScript for all new files. Keep
functions under 50 lines.
```

LAYER 4B — TOOLS

AI Coding Tools

VS Code Extensions

- Roo Code — Code generation, file editing, terminal commands
- Cline — Autonomous coding, browser use, MCP support

Best for: Day-to-day coding. They live in your editor, see your code, and make edits directly.

CLI Tools

- **Claude Code** — Deep codebase understanding
- **OpenCode** — Open-source AI coding assistant
- **Gemini CLI** — Google's CLI for Gemini models
- **OpenClaw** — Open-source CLI AI tool

Best for: Terminal workflows, scripting, server environments. No GUI needed.

Open Manus — Python framework to build custom AI agents for workflows

ESSENTIAL TOOL

📦 Git — Version Control

Git is the **foundation of modern software development**. It tracks every change to your code, lets you experiment safely, and enables collaboration.

🕐 Time Machine

Every saved state is recoverable. Made a mistake? Go back to any previous version instantly.

🧪 Safe Experimentation

Branches let you try new ideas without breaking working code. Merge when ready, discard if not.

👥 Collaboration

Multiple people work on the same project simultaneously. Git merges changes and flags conflicts.

🤖 AI Agent Safety Net

When AI agents edit your code, Git lets you review, accept, or revert every change.

> **ANALOGY**
>
> *Git is like **Ctrl+Z on steroids**. Instead of just undoing your last action, it saves every version of your project forever.*

```
# The commands you'll use 90% of the time git init # Start tracking a project git add . # Stage all changes git commit -m "msg" # Save a checkpoint git branch feat # Create a parallel workspace git push # Upload to GitHub/GitLab git log --oneline # See history
```

LAYER 5

🔗 MCP — Model Context Protocol

MCP is a **standard protocol** that lets AI agents connect to external tools and data sources.

> ANALOGY
>
> ***Before MCP:*** *Every phone had its own charger. Every tool needed custom code.*
> ***After MCP:*** *USB-C. One standard port. Plug in anything — the agent knows how to talk to it.*

📁 File Systems

Read, write, search files on your machine

🗄️ Databases

Query data from SQL, NoSQL, etc.

🌐 Web Browsers

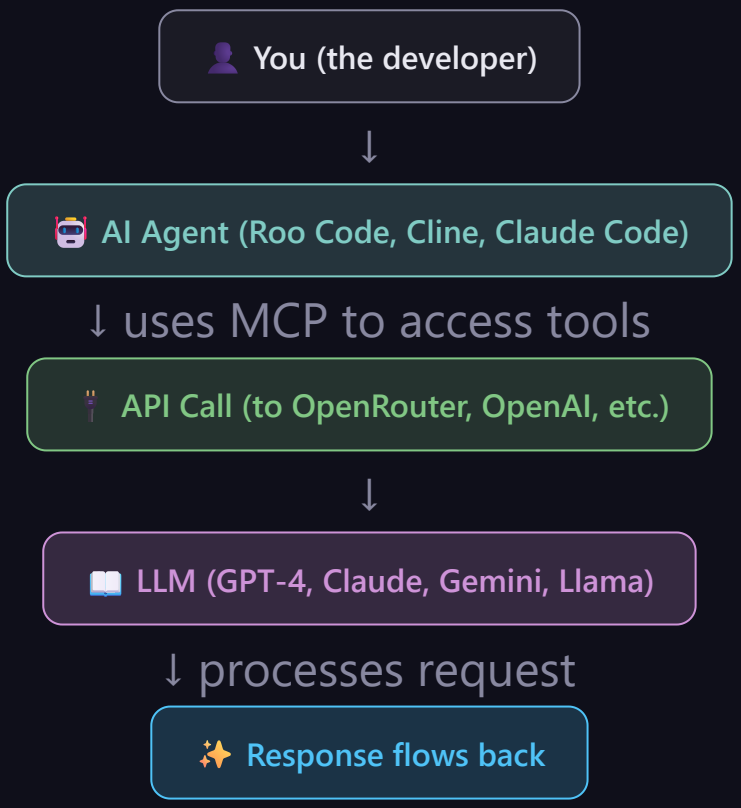
Scrape pages, automate web tasks

📦 Git Repos

Commit, branch, review, and push code

PUTTING IT TOGETHER

The Complete Flow



THE ONE-LINER

*You use an **Agent** → which calls an **API** → which hits an **LLM** → which is a type of **AI**. **MCP** is how the Agent talks to your tools. **Providers** are who hosts the LLM.*

IN PRACTICE

⚡ Example Agent Workflow

Building a REST API endpoint with an AI agent — from task to deployment.

1. You describe the task

↓

2. Agent loads **Skills** (.clinerules, AGENTS.md) + reads codebase via **MCP**

↓

3. Creates branch (git **Tool**), writes code + tests (LLM via API)

↓

4. Runs tests (terminal **Tool**), fixes errors iteratively — guided by **Skills**

↓

5. Commits, pushes, opens PR via **MCP** (git repo connector)

↓

6. You review, approve, merge

REALITY CHECK

⚖️ Pros & Cons of AI-Driven Development

Pros

- 10x faster boilerplate generation
- Agents handle repetitive tasks
- Instant test writing & debugging
- Context-aware code suggestions
- Works while you focus on architecture

Cons

- Can introduce subtle bugs
- Overconfident in wrong answers
- Token costs add up on large codebases
- Requires human review — never trust blindly
- Context window limits on huge projects

ADVANCED PATTERNS

🔀 Sub-Agent Driven Development

Instead of one agent doing everything, a **main agent delegates tasks to specialized sub-agents** that work in parallel.

🤖 Main Agent (Orchestrator)

Receives your high-level task, breaks it down, dispatches sub-agents, and merges results.

⚡ Parallel Execution

Sub-agents run simultaneously on independent tasks. Frontend, backend, tests — all at once.

🌳 Git Worktrees

Each sub-agent gets its own isolated branch + working directory. No conflicts, no overwrites.

🔒 Isolation & Safety

If a sub-agent breaks something, only its worktree is affected. Main branch stays clean.

```
# Git worktrees — each sub-agent gets its own git worktree add ../feat-auth origin/feat-auth # Auth agent git worktree add ../feat-api origin/feat-api # API agent git worktree add ../feat-tests origin/feat-tests # Test agent # Each agent works independently in parallel # When done: review, merge, clean up git worktree remove ../feat-auth
```

ANALOGY

Single agent *= one contractor doing plumbing, electrical, and painting sequentially.*
Sub-agents + worktrees *= a team of specialists working in parallel, each in their own room.*

BEST PRACTICES

🛡️ Best Practices for Using Agents & Chat

AI agents are powerful but need **guardrails**. These habits separate productive users from those who get burned.

🎯 Be Specific

"Add a POST /users endpoint with validation" beats "make a user API." Clear prompts = clear results.

🔍 Always Review Changes

Never blindly accept agent output. Read diffs, run tests, understand what changed before committing.

📦 Small Tasks, Not Monoliths

Break big requests into steps. "Add auth" → "Create user model" → "Add login route" → "Add middleware."

🔒 Protect Secrets

Never paste API keys, passwords, or tokens into chat. Use environment variables and .env files.

GOLDEN RULE

*Treat an AI agent like a **fast but inexperienced junior developer**. Give clear instructions, review their work, and never let them deploy unsupervised.*

BEST PRACTICES — CONTEXT

🧠 Context Management

Every agent session has a **context window** — a finite "memory" that fills up as you chat.

⚠️ What Happens When Context Is Full

The agent drops the oldest messages. It loses your original instructions, earlier decisions, and the reasoning behind them.

✂️ Trim the Conversation

Start a new session for a new task. Don't let one chat grow forever. Summarize what was decided, then reset.

📌 Pin Key Information

Use AGENTS.md, .clinerules, or pinned messages to store critical info. The agent reads these every session.

🗂️ Use Sub-Agents for Big Tasks

Instead of one long conversation, dispatch sub-agents for independent work. Each gets a fresh context window.

> **ANALOGY**
>
> *Context window is like a **whiteboard**. When it's full, you have to erase the oldest stuff to make room.*

```
// Signs your context is overloaded → Agent repeats things you already told it → It ignores earlier instructions or constraints → Responses become generic or off-topic → It "forgets" decisions made 20+ messages ago // Fix: summarize + reset "Here's what we've decided so far: [summary]. Start a fresh session with this context."
```

BEST PRACTICES — PROMPTS

Effective Prompt Patterns

Do This

- Provide context: "In this Express app, add..."
- Specify output format: "Return TypeScript with JSDoc"
- Set constraints: "Max 30 lines, no external deps"
- Ask for reasoning: "Explain before you code"
- Iterate: "Now add error handling"

Avoid This

- Vague requests: "Fix the app"
- Assuming correctness without testing
- Pasting entire codebases in one prompt
- Letting agents modify production data
- Skipping git commits before agent sessions

```
// Good prompt structure Context: "Express + TypeScript REST API" Task: "Add POST /api/users with email validation" Constraints: "Use Zod for validation, max 40 lines" Output: "Route file + one test case" Rule: "Explain your approach before coding"
```

TOOL SPOTLIGHT

🦞 OpenClaw — Open-Source CLI AI Agent

OpenClaw is an open-source CLI-based AI agent that brings sub-agent orchestration, tool use, and MCP support to your terminal.

🖥️ Terminal-First

Lives in your shell. Pipe output, chain commands, script workflows. Perfect for server environments and CI/CD.

🔀 Sub-Agent Orchestration

Spawn independent agents for parallel tasks. Each gets its own context, tools, and workspace.

📱 Telegram Bot Integration

Connect OpenClaw to Telegram — interact with your AI agent from your phone. Send tasks, get results, trigger workflows from anywhere.

🔗 MCP Support

Connect to file systems, databases, browsers, and custom tools through the standard MCP protocol.

OPENCLAW — IN ACTION

🦞 OpenClaw Use Cases

USE CASE	HOW IT WORKS	WHY OPENCLAW
Code Review	Agent reads PR diff, checks for bugs, suggests improvements	CLI-native — runs in CI pipelines
Migration Scripts	"Convert all JS files to TypeScript" — agent plans and executes	Sub-agents handle files in parallel
Data Pipeline	Scrape → clean → load via MCP-connected database	Terminal piping + MCP tools
Infrastructure	Generate Terraform configs from natural language descriptions	No GUI needed — SSH into any server
Documentation	Read codebase via MCP, generate API docs, update README	Batch process across multiple repos

WHEN TO CHOOSE OPENCLAW

*You're comfortable in the terminal, need **automation at scale**, or work in environments without a GUI (servers, containers, CI/CD).*

CHEAT SHEET

Quick Reference Card

TERM	ONE-LINE DEFINITION	ANALOGY
AI	Machines that simulate intelligence	"Vehicles" (the category)
LLM	Text prediction engine trained on massive data	"Engine" (one type of vehicle)
Token	Unit of text the LLM processes	"Words" (roughly)
Context Window	How much text the LLM can process at once	"Working memory"
API	Program-to-program communication	"Phone call"
Provider	Company that hosts the LLM	"Phone company"
Agent	AI that can act autonomously with tools	"Employee"
MCP	Standard protocol for agent-tool connections	"USB-C"

Questions?

From AI fundamentals to agents, MCP, and modern dev tools — you now know how it all connects.

AI & LLMs | APIs | Providers | Agents | MCP | Git | Antigravity | OpenCode Desktop

rupan@mgtechsoft.com